

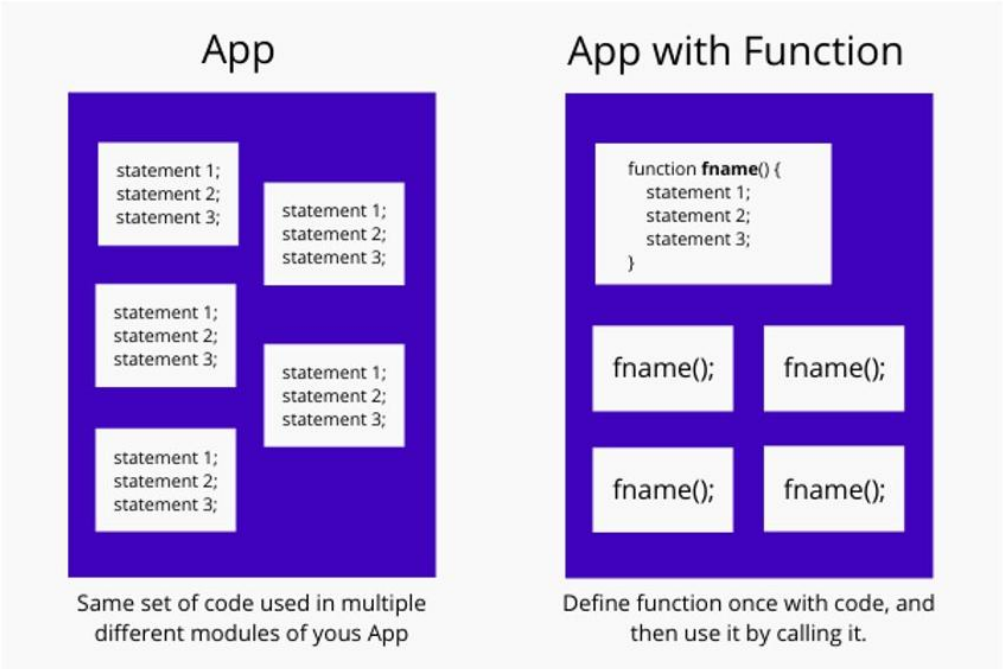
TECHIN

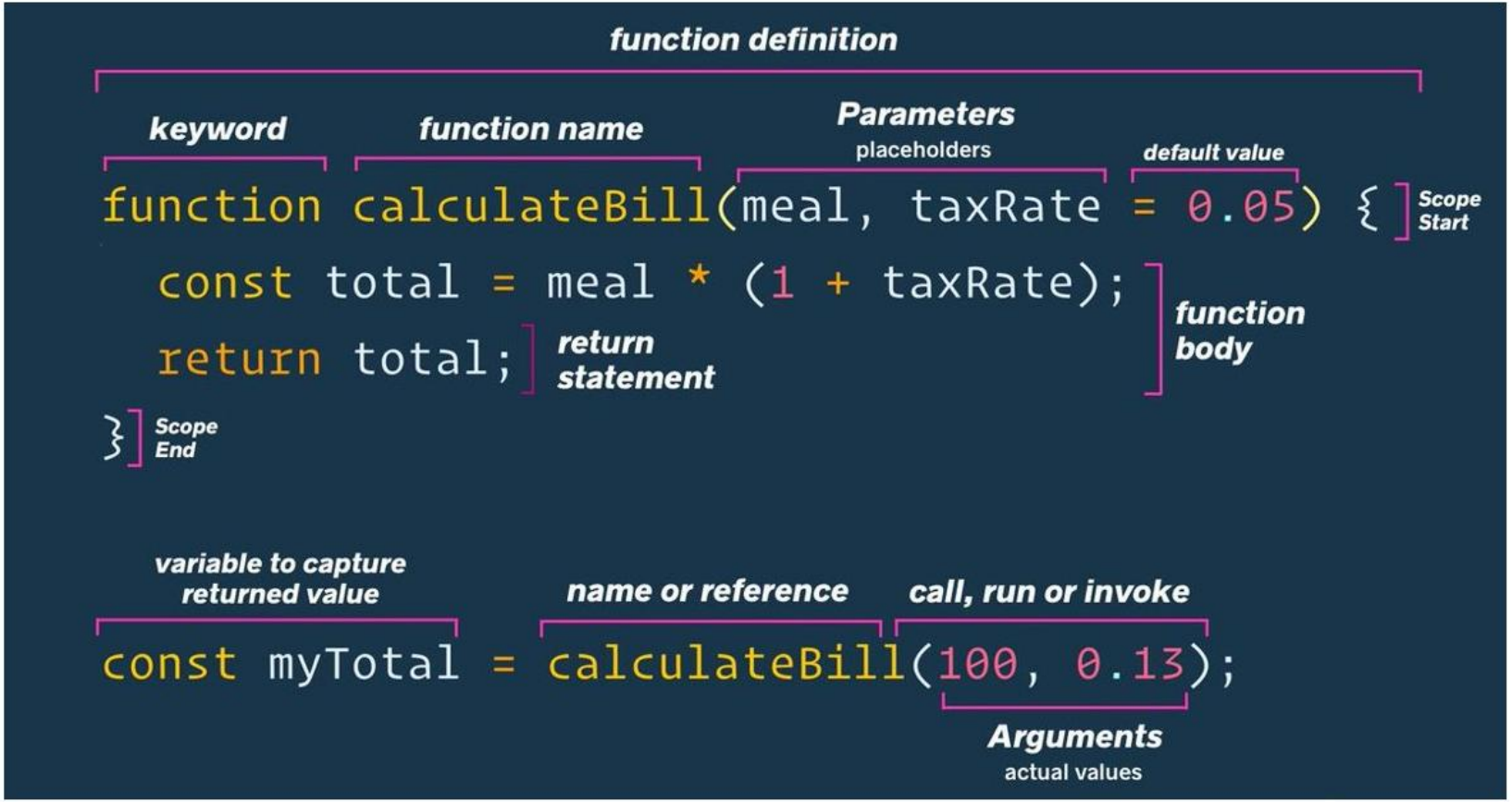
JavaScript

Funkcijos

JavaScript funkcijos

- **Funkcijos – tai kodo blokai, kurie veikia tik juos iškvietus.**
- **Pasikartojantį kodą vieną kartą parašome, daug kartų įvykdome.**




function definition
keyword
function name
Parameters
placeholders
default value
function calculateBill(meal, taxRate = 0.05) {
Scope Start
const total = meal * (1 + taxRate);
return total;
return statement
function body
}
Scope End
variable to capture returned value
name or reference
call, run or invoke
const myTotal = calculateBill(100, 0.13);
Arguments
actual values

JavaScript funkcijos

Tipai:

1. Function declaration
2. Function expression
3. Arrow functions



JavaScript funkcijos

Function declaration

```
function sum(a, b) {
  let result = a + b;

  return result;
}
```

Funkcijos iškvietimas: sum(5, 17);

Jei parašysime tik sum – atspausdins funkciją.

JavaScript funkcijos

Function expression

```
1  let sum = function(a, b) {
2    let result = a + b;
3
4    return result;
5  };
```

Funkcijos iškvietimas: sum(5, 17);

Jei parašysime tik sum – atspausdins funkciją.



JavaScript funkcijos

Arrow functions

```
1  // expression at the right side
2  let sum = (a, b) => a + b;
3
4  // or multi-line syntax with { ... }, need return here:
5  let sum = (a, b) => {
6    // ...
7    return a + b;
8  }
9
10 // without arguments
11 let sayHi = () => alert("Hello");
12
13 // with a single argument
14 let double = n => n * 2;
```



JavaScript callback funkcijos

```
function isOdd(number) {
  return number % 2 != 0;
}
function isEven(number) {
  return number % 2 == 0;
}

function filter(numbers, fn) {
  let results = [];
  for (const number of numbers) {
    if (fn(number)) {
      results.push(number);
    }
  }
  return results;
}
let numbers = [1, 2, 4, 7, 3, 5, 6];

console.log(filter(numbers, isOdd));
console.log(filter(numbers, isEven));
```

Tai tokios funkcijos, kurios perduodamos į kitas funkcijas kaip argumentas.

- **Funkcijos isOdd ir isEven yra vykdomos sinchroniškai, nes yra įvykdomos kartu su filter funkcija.**
- **Funkcija filter yra vadinama high-order function (HOF).**



JavaScript callback funkcijos

- Callback funkcijos gali būti vykdomos ir asinchroniškai, t.y. callback funkcija yra vykdoma atskirai ir įvykdoma vėliau nei higher-order funkcija.
- Šis vykdymo būdas reikalingas tada, kai callback vykdymas užtrunka, pvz. iš serverio siunčiami duomenys.

```
function callBackTech(callback, tech) {
    console.log("Calling callBackTech!");
    if (callback) {
        setTimeout(() => callback(tech), 2000)
    }
    console.log("Calling callBackTech finished!");
}

function logTechDetails(tech) {
    if (tech) {
        console.log("The technology used is: " + tech);
    }
}

callBackTech(logTechDetails, "HTML5");
```

Output

```
Calling callBackTech!                          debugger eval code:2:13
Calling callBackTech finished!                 debugger eval code:6:13
← undefined
The technology used is: HTML5                  debugger eval code:11:17
```



JavaScript funkcijos

- **Funkcijos nesuveikia, kol nėra iškviečiamos. Bet yra tokių funkcijų, kurios gali iškviesti pačios save.**
- **Tokios gali būti tik function expression.**

Jei funkcija be parametrų:

```
(function () {
  var x = "Hello!!";  // I will invoke myself
})();
```

Jei funkcija su parametrais:

```
(function (message) {
  document.getElementById("demo").innerHTML = message;
})("Hello, hello");
```

JavaScript funkcijos
Nested

- Galime rašyti sudėtines funkcijas.
- Viduje esanti funkcija mato jos išorėje esančius kintamuosius.

1 pavyzdys:

```
function init() {
  var name = 'Mozilla'; // name is a local variable created by init
  function displayName() { // displayName() is the inner function, a closure
    alert(name); // use variable declared in the parent function
  }
  displayName();
}
init();
```



JavaScript funkcijos
Nested

- Galime rašyti sudėtines funkcijas.
- Viduje esanti funkcija mato jos išorėje esančius kintamuosius.

2 pavyzdys:

```
1   function sayHiBye(firstName, lastName) {
2     // helper nested function to use below
3     function getFullName() {
4       return firstName + " " + lastName;
5     }
6
7     alert("Hello, " + getFullName());
8     alert("Bye, " + getFullName());
9   }
10
11  sayHiBye("Jonas", "Jonaitis");
```

JavaScript metodai

- JavaScript turi funkcijas ir metodus.
- Metodai yra tos pačios funkcijos, bet susietos su objektais arba duomenų tipais. Skiriasi iškvietimas. Metodai iškviečiami su taškeliu.
- Pavyzdyje – metodas susietas su duomenų tipu String.

```
1  const name = 'shaun';
2
3  // functions
4
5  const greet = () => 'hello';
6
7  let resultOne = greet();
8  console.log(resultOne);
9
10 // methods
11
12 let resultTwo = name.toUpperCase();
13 console.log(resultTwo);
14
```